

Funktionen `main()`

Som vi redan tidigare såg i programmet *Hello world!* finns det en speciell funktion som heter `main()`. Denna funktion är speciell på det sättet att den är alltid den första funktionen som utförs i ett C/C++-program. Denna funktion kan sedan i sin tur anropa andra funktioner och på så vis bygga upp den egentliga funktionaliteten i programmet. Det finns inga undantag till detta, `main()` är alltid den första funktionen. När funktionen `main()` avslutas, avslutas även hela programmet. Vi ska i fortsättningen alltid använda `main()` i exempel där det är möjligt, för att göra dem mera kompletta.

Funktionen `main()` kan inte heta något annat än just `main()`, inga andra stavningar tillåts. Om kompilatorn inte hittar denna funktion när den kompilerar ett program fås en varning eller ett fel. Ett program med två funktioner som anropas från `main()` kan se ut som följande:

```
#include <iostream>

float RaknaArea (float Sida1, float Sida2) {
    // beräkna och returnera en area av en rektangel
    return Sida1 * Sida2;
}

void SkrivSvar (float Area) {
    cout << "Areal är: " << Area << endl;
}

int main () {
    // beräkna arean
    float Area = RaknaArea ( 5.5, 10 );

    // skriv ut svaret
    SkrivSvar ( Area );
}
```

Här har vi nu två skilda funktioner som anropas från `main()`. En liten förenkling som kunde göras i `main()` är att kombinera areaberäkningen med utskriften. Eftersom `RaknaArea()` returnerar en `float`, och `SkrivSvar()` vill ha en `float` som parameter kan man skriva om `main()` på detta sätt:

```
int main () {
    // beräkna arean och skriv ut svaret
    SkrivSvar ( RaknaArea ( 5.5, 10 ) );
}
```

Nu använder vi resultatet från `RaknaArea()` direkt och skickar det till `SkrivSvar()` utan använda en temporär variabel som mellanlagring.

Returvärde från `main()`

Som vi sett deklaras `main()` alltid med returvärdet `int`, även om vi aldrig utför någon `return`-sats. Detta returvärde skall om det ges vara ett positivt heltal som skall ange hur programmets körning lyckades. Värdet 0 indikerar en lyckad körning utan fel, medan andra värden indikerar fel av olika typ. Det system som kör programmet kan således få reda på hur programmets exekvering gick. Om inget returvärde ges explicit placera kompilatorn automatiskt en sats `return 0` sist i programmet.

Alltså om man inte manuellt specificerar returvärdet från `main()` fås alltid värdet 0. Detta gäller dock endast funktionen `main()`.

Prototyper

I exemplen ovan har vi alltid haft `main()` sist i programmet. Vad händer om man istället flyttar `main()` först, och placerar `RaknaArea()` och `SkrivSvar()` efter? Man skulle ju tycka att det inte vore någon skillnad överhuvudtaget, men det är tyvärr en skillnad. C++-kompilatorer är för det mesta ganska lata, och kräver att alla symboler, variabler och funktioner måste vara definierade före de används. Då kompilatorn läser ett program med `main()` först, kommer den att stöta på funktionerna `RaknaArea()` och `SkrivSvar()` som inte ännu är definierade, eftersom de definieras först några rader senare! Allting måste alltså vara definierat före det används. Tänk om man har en situation med två funktioner som anropar varandra. I det fallet går det helt enkelt inte att definiera båda funktionerna före de används. Det finns en omväg runt detta dilemma, och det är något som kallas *prototyper*. En prototyp är en definition som är menad för kompilatorn och berättar hur en funktion ser ut. Prototypen innehåller ingen kod, endast funktionens namn, returvärde och parametrar. Allmänt ser det ut så här:

```
returtyp funktionsnamn (parametertyp1 parameter1, ... );
```

En funktionsprototyp har ingen kropp innesluten mellan `{` och `}`. När kompilatorn stöter på en dylik rad vet den hur en viss funktion ser ut, och kan använda den redan innan den definieras. Kompilatorn vet även att själva koden för funktionen "kommer senare". Vi kan nu skriva om vår areaberäkning med prototyper så att `main()` är först:

```
#include <iostream>

// prototyper för våra två funktioner
float RaknaArea (float Sida1, float Sida2);
void SkrivSvar (float Area);

int main () {
    // beräkna arean
    float Area = RaknaArea ( 5.5, 10 );

    // skriv ut svaret
    SkrivSvar ( Area );
}

float RaknaArea (float Sida1, float Sida2) {
    // beräkna och returnera en area av en rektangel
    return Sida1 * Sida2;
}

void SkrivSvar (float Area) {
    cout << "Areal är: " << Area << endl;
}
```

Notera de två prototyperna överst. De *måste* stämma överens med de parametrar och returvärden som sedan egentligen används av funktionerna. Om prototyper ges för funktioner som sedan aldrig skrivs kommer kompilatorn att ge ett felmeddelande. Den "magiska" raden `#include <iostream>` kommer att behandlas senare (se [Kapitel 10](#)), och nu räcker det med att raden syftar till en fil som innehåller prototyp-deklarationer för de I/O-funktioner vi använder (`cout`).

Parametertyper

[Upp](#)Parametrar till `main()`